

第四步完成报告：MicroDopplerModulator 核心模块

1. 第四步在项目中的位置

SionnaEM 项目的核心目标是验证 Micro-Doppler 效应能否集成入 Sionna RT 信道仿真平台。项目分六步推进：



前三步证明了物理可行性（“能不能做”），第四步将验证脚本转化为可复用的软件模块（“怎么做”），是项目从“验证原型”到“可交付代码”的关键转折点。

2. 产出物

2.1 核心文件

文件	行数	功能
micro_doppler_modulator.py	~370	Micro-Doppler 调制引擎
verify_modulator.py	~230	26 项系统验证

2.2 两个核心类

UAVMicroDopplerConfig — 参数配置数据类

25 个字段的 @dataclass，封装了完整的四旋翼 UAV 仿真参数：

- 电磁参数：**载波频率 → 自动推导波长
- 几何参数：**叶片半径、旋翼臂长、旋翼数量、叶片数量
- 动力学参数：**旋转频率、旋翼初始相位

- **RCS 权重**: 机身幅度、叶片幅度
- **雷达参数**: 雷达位置、工作模式（单基地/双基地）
- **运动参数**: 机身初始位置、平移速度
- **仿真参数**: 采样率、观测时长、信噪比

关键特性: `__post_init__` 自动推导 8 个派生量（波长、角频率、散射点数量、调制指数 β 、峰值频偏、旋翼位置、叶片相位），减少用户计算负担。

MicroDopplerModulator — 调制引擎

7 个公共方法 + 2 个私有方法:

方法	输入	输出	用途
<code>scatterer_positions(t)</code>	时间序列	<code>[n_scatterers, n_t, 3]</code> 3D 轨迹	几何建模
<code>generate_received_signal(t)</code>	时间序列	<code>[n_t]</code> 复基带信号	独立仿真
<code>modulate_cir(a, tau, t)</code>	Sionna 静态 CIR	时变 CIR 序列	RT 集成
<code>stft_spectrogram(signal)</code>	复信号	零中心 STFT 谱图	时频分析
<code>detect_harmonics(signal)</code>	复信号	谐波列表	特征提取
<code>summary()</code>	—	参数摘要	诊断输出

3. 在项目中的用途

3.1 替代三步验证脚本的散落逻辑

前三步中，散射点建模、相位调制、STFT 分析的代码分散在三个独立脚本中，存在大量重复。
`MicroDopplerModulator` 将这些逻辑统一封装，消除了代码重复：

之前: `baseline_doppler.py`

→ 硬编码线性运动

`micro_doppler_single_scatterer.py` → 硬编码单散射点旋转

`micro_doppler_quadrotor_spectrogram.py` → 硬编码 9 散射点模型

之后：MicroDopplerModulator(config) → 配置驱动，一套代码覆盖所有场景

3.2 接入 Sionna RT 管道的标准接口

modulate_cir() 方法是模块与 Sionna RT 之间的桥梁：

```
# 标准 Sionna RT 流程
scene = load_scene(sionna.rt.scene.etoile)
paths = solver(scene, ...) # 射线追踪（仅一次）
a_static, tau_static = paths.cir(...) # 静态 CIR

# 第四步新增：Micro-Doppler 调制
config = UAVMicroDopplerConfig(...)
modulator = MicroDopplerModulator(config)
a_dynamic, tau_dynamic = modulator.modulate_cir( # 时变 CIR
    a_static, tau_static, t_vec
)

# 后续：CFR 生成、信道估计、检测算法等
```

关键特性：RT 引擎只调用一次（处理静态场景），时变调制在 CIR 后处理阶段叠加。RT 引擎修改量为 **0** 行。

3.3 支持两种工作模式

模式	方法	依赖 Sionna RT	适用场景
独立模式	generate_received_signal()	否	快速原型、参数扫描、算法验证
集成模式	modulate_cir()	是	真实场景仿真、多径分析、ISAC

3.4 为第五步和第六步提供基础

- **第五步（CIR 管道集成）**：直接调用 modulate_cir() 处理 Sionna 的 Paris etoile 场景 CIR，对比独立模式验证多径影响
 - **第六步（论文对标）**：通过修改 UAVMicroDopplerConfig 参数快速重现论文中的不同 UAV 配置（不同 R、f_rot、旋翼数）
-

4. 正确性验证（26/26 通过）

4.1 验证框架

`verify_modulator.py` 从三个层级验证模块正确性：

- Tier A (9 项)：与 Step 3 交叉验证
 - 相同参数 → 相同谱图、相同 β 、相同 Doppler、相同谐波
- Tier B (8 项)：真实 Sionna RT CIR 集成
 - 加载 Paris etoile 场景 → RT 求解 → `modulate_cir()` → 验证频谱扩展
- Tier C (9 项)：物理一致性 + 边界条件
 - $\beta \propto R$ 、 $v=0$ → 相位恒定、单/双基地 β 比值、NaN/Inf 检查

4.2 关键验证结果

验证项	预期值	实测值	结论
调制指数 β	176.1 rad	176.1 rad	误差 0.02%
机身 Doppler（单基地）	-934.0 Hz	-934.0 Hz	误差 0.00%
$\beta \propto R$ 线性度	β/R 恒定	$\sigma/\mu = 0$	精确线性
单/双基地 β 比值	2.0	2.0	精确
STFT 频率分辨率	f_s/N_{win}	一致	匹配
<code>modulate_cir</code> 输出维度	Sionna 规范	正确	—
旋转平面约束	z 恒定	$< 1e-12$ m	—
信号有效性	无 NaN/Inf	通过	—

5. 与 Step 3 的对比

维度	Step 3 脚本	Step 4 模块
参数管理	硬编码常量	<code>UAVMicroDopplerConfig</code> dataclass
代码复用性	单文件不可导入	<code>from micro_doppler_modulator import ...</code>
Sionna RT 集成	无	<code>modulate_cir()</code> 方法

维度	Step 3 脚本	Step 4 模块
可配置性	修改源码	修改 config 对象
测试覆盖	无	26 项验证 + 9 项单元测试
文档	仅有 docstring	<code>summary()</code> + 完整类型注解

6. 技术要点

6.1 单基地往返 Doppler 公式

模块中单基地雷达的 Doppler 频移使用往返公式：

$$f_d = -\frac{2 f_c}{c} \cdot v_{\text{radial}}$$

而非单程公式 $f_d = -f_c \cdot v_{\text{radial}} / c$ 。验证中实测 -934.0 Hz 与往返理论值 -934.0 Hz 完全一致（误差 0.00%）。

6.2 STFT 频率轴零中心对齐

`stft_spectrogram()` 正确实现了 `fftshift`：

```
f = np.fft.fftshift(np.fft.fftfreq(n_win, 1 / fs)) # 频率轴 shift
Z_shifted = np.fft.fftshift(Z, axes=0)           # 数据 shift
```

修复了前三步脚本中频率轴与数据不对齐的 bug。

6.3 机身主导效应的物理正确性

`modulate_cir()` 的输出相位被机身主导（ $A_{\text{body}} = 1.0 \gg A_{\text{blade}} = 0.12$ ）。这并非 bug，而是物理正确行为——真实雷达中机身 RCS 远大于单个叶片。Micro-Doppler 特征体现在频谱扩展上（机身能量集中于 DC，叶片能量展布于 $\pm 17.6 \text{ kHz}$ 带宽），验证中确认了这一点。

7. 后续工作

第四步为以下工作奠定了基础：

- **第五步：**在多场景（Paris etoile 多径、floor_wall 简单场景）中验证 `modulate_cir()` 的性能和准确性

- **第六步：**通过修改 `UAVMicroDopplerConfig` 参数快速对标论文中的不同 UAV 配置
 - **向何老师汇报：**第四步的 `summary()` 方法和验证结果可直接用于展示
-

附录：快速开始

```
from micro_doppler_modulator import UAVMicroDopplerConfig, MicroDopplerModulator
import numpy as np

# 1. 创建配置
config = UAVMicroDopplerConfig(
    carrier_freq=28e9,
    blade_radius=0.15,
    rotation_freq=100.0,
    body_velocity=(5.0, 0.0, 0.0),
)

# 2. 创建调制器
mod = MicroDopplerModulator(config)
print(mod.summary())

# 3. 生成信号
t = np.arange(int(config.sampling_rate * config.obs_duration)) / config.sampling_rate
signal = mod.generate_received_signal(t)

# 4. 谱图分析
f, t_s, S_dB = mod.stft_spectrogram(signal)
harmonics = mod.detect_harmonics(signal)
print(f"Detected {len(harmonics)} harmonics of f_rot={config.rotation_freq} Hz")
```